

Relational Algebra and SQL Queries on Employee Database

Employee Database Schema

- **employee**(*person_name*, street, city)
- **works**(*person_name*, *company_name*, salary)
- **company**(*company_name*, city)

Bank Database Schema

- **branch**(*branch_name*, branch_city, assets)
- **customer**(ID, customer_name, customer_street, customer_city)
- **loan**(loan_number, branch_name, amount)
- **borrower**(ID, loan_number)
- **account**(account_number, branch_name, balance)
- **depositor**(ID, account_number)
- **employee**(emp_id, name, city, salary, ...)

University Database Schema

- **classroom**(*building*, room number, capacity)
- **department**(*dept name*, building, budget)
- **course**(*course id*, title, dept name, credits)
- **instructor**(*ID*, name, dept name, salary)
- **section**(*course id*, sec id, semester, year, building, room number, time slot id)
- **teaches**(*ID*, course id, sec id, semester, year)
- **student**(*ID*, name, dept name, tot cred)
- **takes**(*ID*, course id, sec id, semester, year, grade)
- **advisor**(*s ID*, *i ID*)
- **time slot**(*time slot id*, day, start time, end time)
- **prereq**(*course id*, prereq id)

Insurance Database Schema

- person(driver_id, name, address)
- car(license_plate, model, year)
- accident(report_number, year, location)
- owns(driver_id, license_plate)
- participated(report_number, license_plate, driver_id, damage_amount)

Queries on Employee Database

Exercise: Queries on Employee Database

Express each of the following queries in both Relational Algebra and SQL:

- Find the name of each employee who lives in city “Miami”.
- Find the name of each employee whose salary is greater than \$100,000.
- Find the name of each employee who lives in “Miami” and whose salary is greater than \$100,000.

a) Employees living in Miami

Relational Algebra:

$$\pi_{\text{name}}(\sigma_{\text{city}=\text{'Miami'}}(\text{employee}))$$

SQL Query:

```
SELECT name
FROM employee
WHERE city = 'Miami';
```

b) Employees with salary greater than \$100,000

Relational Algebra:

$$\pi_{\text{name}}(\sigma_{\text{salary} > 100000}(\text{employee}))$$

SQL Query:

```
SELECT name
FROM employee
WHERE salary > 100000;
```

c) Employees living in Miami and salary greater than \$100,000

Relational Algebra:

$$\pi_{\text{name}}(\sigma_{\text{city} = \text{'Miami'} \wedge \text{salary} > 100000}(\text{employee}))$$

SQL Query:

```
SELECT name
FROM employee
WHERE city = 'Miami' AND salary > 100000;
```

Bank Database Queries

Exercise: Queries on Bank Database

Express each of the following queries in both Relational Algebra and SQL:

- Find the name of each branch located in “Chicago”.
- Find the ID of each borrower who has a loan in branch “Downtown”.

a) Branches located in Chicago

Relational Algebra:

$$\pi_{\text{branch_name}}(\sigma_{\text{branch_city} = \text{'Chicago'}}(\text{branch}))$$

SQL Query:

```
SELECT branch_name
FROM branch
WHERE branch_city = 'Chicago';
```

b) Borrowers with loans in Downtown branch

Relational Algebra:

$$\pi_{ID}(\sigma_{\text{branch_name}='Downtown'}(\text{borrower} \bowtie \text{loan}))$$

(natural join on loan_number)

SQL Query:

```
SELECT DISTINCT b.ID
FROM borrower b
JOIN loan l ON b.loan_number = l.loan_number
WHERE l.branch_name = 'Downtown';
```

Additional Employee Database Queries with Explanation

Exercise: Additional Queries on Employee Database

Express each of the following queries in both Relational Algebra and SQL:

- Find the ID and name of each employee who does **not** work for “BigBank”.
- Find the ID and name of each employee who earns at least as much as **every** employee in the database.

a) Employees not working for BigBank

Explanation: To find employees who do **not** work for BigBank, we first find those who **do** work for BigBank. Then we subtract this set from the set of all employees, using the set difference operation.

Relational Algebra:

$$\pi_{ID, \text{name}}(\text{employee}) - \pi_{ID, \text{name}}(\sigma_{\text{company_name}='BigBank'}(\text{employee} \bowtie \text{works}))$$

SQL Query:

```
SELECT ID, name
FROM employee
WHERE ID NOT IN (
    SELECT ID
    FROM works
    WHERE company_name = 'BigBank'
);
```

b) Employees earning at least as much as every employee

Explanation: We find employees for whom no other employee earns a higher salary. Since we need to compare employees' salaries, we reference the employee relation twice with renaming.

Relational Algebra:

$$\pi_{ID, name}(e) \quad \text{where} \quad e \in \text{employee} \wedge \neg \exists e' \in \rho_{e'}(\text{employee})(e'.salary > e.salary)$$

Or explicitly:

$$\pi_{ID, name}(\text{employee}) - \pi_{e.ID, e.name}(\sigma_{e.salary < e'.salary}(\rho_e(\text{employee}) \times \rho_{e'}(\text{employee})))$$

SQL Query:

```
SELECT e1.ID, e1.name
FROM employee e1
WHERE NOT EXISTS (
    SELECT 1
    FROM employee e2
    WHERE e2.salary > e1.salary
);
```

Bank Database Keys Analysis

Question

- What are the appropriate primary keys?
- Given your choice of primary keys, identify appropriate foreign keys.

Solution

a) Primary Keys:

- **branch:** `branch_name` (uniquely identifies each branch)
- **customer:** `customer_name` (unique for each customer)
- **loan:** `loan_number` (unique loan identifier)
- **account:** `account_number` (unique account identifier)
- **borrower:** Composite key (`ID`, `loan_number`) (associates customers with loans)
- **depositor:** Composite key (`ID`, `account_number`) (associates customers with accounts)

b) Foreign Keys:

- In **loan**, `branch_name` references `branch(branch_name)`.
- In **account**, `branch_name` references `branch(branch_name)`.
- In **borrower**, `ID` references `customer(ID)` and `loan_number` references `loan(loan_number)`.
- In **depositor**, `ID` references `customer(ID)` and `account_number` references `account(account_number)`.

Queries on Employee Database

Queries

- Find the ID and name of each employee who works for “BigBank”.
- Find the ID, name, and city of residence of each employee who works for “Big-Bank”.
- Find the ID, name, street address, and city of residence of each employee who works for “BigBank” and earns more than \$10,000.
- Find the ID and name of each employee in this database who lives in the same city as the company for which he or she works.

a) Employees working for BigBank

Relational Algebra:

$$\pi_{\text{person_name}, \text{ID}}(\sigma_{\text{company_name}='BigBank'}(\text{works}))$$

SQL Query:

```
SELECT person_name, ID
FROM works
WHERE company_name = 'BigBank';
```

b) Employees working for BigBank with city

Relational Algebra:

$$\pi_{\text{ID}, \text{person_name}, \text{city}}(\sigma_{\text{company_name}='BigBank'}(\text{works}) \bowtie \text{employee})$$

SQL Query:

```
SELECT w.ID, w.person_name, e.city
FROM works w JOIN employee e ON w.person_name = e.
    person_name
WHERE w.company_name = 'BigBank';
```

c) Employees working for BigBank with address and salary greater than 10000

Relational Algebra:

$$\pi_{\text{ID}, \text{person_name}, \text{street}, \text{city}}(\sigma_{\text{company_name}='BigBank' \wedge \text{salary} > 10000}(\text{works} \bowtie \text{employee}))$$

SQL Query:

```
SELECT w.ID, w.person_name, e.street, e.city
FROM works w JOIN employee e ON w.person_name = e.
    person_name
WHERE w.company_name = 'BigBank' AND w.salary > 10000;
```

d) Employees living in same city as company they work for

Relational Algebra:

$$\pi_{w.ID, w.person_name}(\sigma_{e.city=c.city}((works\ w) \bowtie employee\ e \bowtie company\ c))$$

where the joins are on

$$w.person_name = e.person_name \quad \text{and} \quad w.company_name = c.company_name$$

SQL Query:

```
SELECT w.ID, w.person_name
FROM works w
JOIN employee e ON w.person_name = e.person_name
JOIN company c ON w.company_name = c.company_name
WHERE e.city = c.city;
```

Queries on Bank Database

Queries

- Find each loan number with a loan amount greater than \$10,000.
- Find the ID of each depositor who has an account with a balance greater than \$6,000.
- Find the ID of each depositor who has an account with a balance greater than \$6,000 at the “Uptown” branch.

a) Loan numbers with amount greater than 10000

Relational Algebra:

$$\pi_{loan_number}(\sigma_{amount > 10000}(loan))$$

SQL Query:

```
SELECT loan_number
FROM loan
WHERE amount > 10000;
```


b) Depositor IDs with account balance greater than 6000

Relational Algebra:

$$\pi_{ID}(\sigma_{balance > 6000}(account) \bowtie depositor)$$

SQL Query:

```
SELECT DISTINCT d.ID
FROM depositor d
JOIN account a ON d.account_number = a.account_number
WHERE a.balance > 6000;
```

c) Depositor IDs with account balance greater than 6000 at Uptown branch

Relational Algebra:

$$\pi_{ID}(\sigma_{balance > 6000 \wedge branch_name = 'Uptown'}(account) \bowtie depositor)$$

SQL Query:

```
SELECT DISTINCT d.ID
FROM depositor d
JOIN account a ON d.account_number = a.account_number
WHERE a.balance > 6000 AND a.branch_name = 'Uptown';
```

Queries on University Database

Queries

- Find the ID and name of each instructor in the Physics department.
- Find the ID and name of each instructor in a department located in the building "Watson".
- Find the ID and name of each student who has taken at least one course in the "Comp. Sci." department.
- Find the ID and name of each student who has taken at least one course section in the year 2018.
- Find the ID and name of each student who has not taken any course section in the year 2018.

a) Instructors in Physics department

Relational Algebra:

$$\pi_{ID, name}(\sigma_{dept_name='Physics'}(instructor))$$

SQL Query:

```
SELECT ID, name
FROM instructor
WHERE dept_name = 'Physics';
```

b) Instructors in departments located in building "Watson"

Relational Algebra:

$$\pi_{ID, name}(instructor \bowtie_{instructor.dept_name=department.dept_name} \sigma_{building='Watson'}(department))$$

SQL Query:

```
SELECT i.ID, i.name
FROM instructor i
JOIN department d ON i.dept_name = d.dept_name
WHERE d.building = 'Watson';
```

c) Students who took at least one course in "Comp. Sci." department

Relational Algebra:

$$\pi_{ID, name}(student \bowtie (\sigma_{dept_name='Comp. Sci.'}(course) \bowtie takes))$$

SQL Query:

```
SELECT DISTINCT s.ID, s.name
FROM student s
JOIN takes t ON s.ID = t.ID
JOIN course c ON t.course_id = c.course_id
WHERE c.dept_name = 'Comp. Sci.';
```

d) Students who took at least one course section in 2018

Relational Algebra:

$$\pi_{ID,name}(\text{student} \bowtie \sigma_{year=2018}(\text{takes}))$$

SQL Query:

```
SELECT DISTINCT s.ID, s.name
FROM student s
JOIN takes t ON s.ID = t.ID
WHERE t.year = 2018;
```

e) Students who did NOT take any course section in 2018

Relational Algebra:

$$\pi_{ID,name}(\text{student}) - \pi_{ID,name}(\text{student} \bowtie \sigma_{year=2018}(\text{takes}))$$

SQL Query:

```
SELECT s.ID, s.name
FROM student s
WHERE s.ID NOT IN (
    SELECT t.ID
    FROM takes t
    WHERE t.year = 2018
);
```

Queries on University Database

Exercise: Queries on University Database

Express the following query in both Relational Algebra and SQL:

Find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both.

Courses taught in Fall 2009 or Spring 2010

Relational Algebra:

$$\pi_{\text{course_id}}(\sigma_{\text{semester}='Fall'\wedge\text{year}=2009}(\text{section})) \cup \pi_{\text{course_id}}(\sigma_{\text{semester}='Spring'\wedge\text{year}=2010}(\text{section}))$$

SQL Query:

```
SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2009
UNION
SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2010;
```

Queries on University Database

Exercise: Queries on University Database

Express the following query in both Relational Algebra and SQL:

Find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester.

Courses taught in Fall 2009 but not Spring 2010

Relational Algebra:

$$\pi_{\text{course_id}}(\sigma_{\text{semester}='Fall'\wedge\text{year}=2009}(\text{section})) - \pi_{\text{course_id}}(\sigma_{\text{semester}='Spring'\wedge\text{year}=2010}(\text{section}))$$

SQL Query:

```
SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2009
EXCEPT
SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2010;
```

Queries on University Database

Exercise: Queries on University Database

Express the following query in both Relational Algebra and SQL:

Find the names of all instructors in the Physics department, along with the course_id of all courses they have taught.

Solution Variations

Relational Algebra (Query 1):

$$\pi_{\text{instructor.ID, course_id}} \left(\sigma_{\text{dept_name}='Physics'} \left(\sigma_{\text{instructor.ID}=\text{teaches.ID}} (\text{instructor} \times \text{teaches}) \right) \right)$$

Relational Algebra (Query 2):

$$\pi_{\text{instructor.ID, course_id}} \left(\sigma_{\text{instructor.ID}=\text{teaches.ID}} \left(\sigma_{\text{dept_name}='Physics'} (\text{instructor}) \times \text{teaches} \right) \right)$$

Relational Algebra (Query 3):

$$\pi_{\text{instructor.ID, course_id}} \left(\sigma_{\text{dept_name}='Physics' \wedge \text{instructor.ID}=\text{teaches.ID}} (\text{instructor} \times \text{teaches}) \right)$$

SQL Query:

```
SELECT instructor.ID, course_id
FROM instructor
JOIN teaches ON instructor.ID = teaches.ID
WHERE dept_name = 'Physics';
```

Queries on University Salaries

Exercise: Queries on University Salaries

Express the following in Relational Algebra and SQL:

Find the largest salary in the university. Use a two-step process: (1) Find salaries that are less than some other instructor's salary. (2) Subtract them from the set of all salaries to get the largest salary.

Finding the Largest Salary

Step 1 — Salaries less than some other instructor's salary:

$$\pi_{\text{instructor.salary}}(\sigma_{\text{instructor.salary} < d.\text{salary}}(\text{instructor} \times \rho_d(\text{instructor})))$$

Step 2 — Largest salary:

$$\pi_{\text{salary}}(\text{instructor}) - \pi_{\text{instructor.salary}}(\sigma_{\text{instructor.salary} < d.\text{salary}}(\text{instructor} \times \rho_d(\text{instructor})))$$

SQL Query:

```
SELECT salary
FROM instructor
WHERE salary NOT IN (
    SELECT DISTINCT i1.salary
    FROM instructor i1, instructor i2
    WHERE i1.salary < i2.salary
);
```

Queries on University Database

Exercise: Queries on University Database

Express the following query in both Relational Algebra and SQL:

Find all courses taught in both the Fall 2009 semester and the Spring 2010 semester.

Courses taught in both Fall 2009 and Spring 2010

Relational Algebra:

$$\pi_{\text{course_id}}(\sigma_{\text{semester}='Fall' \wedge \text{year}=2009}(\text{section})) \cap \pi_{\text{course_id}}(\sigma_{\text{semester}='Spring' \wedge \text{year}=2010}(\text{section}))$$

SQL Query:

```
SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2009
INTERSECT
SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2010;
```

Queries on Instructor and Course Database

Exercise: Queries on Instructor and Course Database

Express the following query in both Relational Algebra and SQL:

Find the names of all instructors in the Comp. Sci. department together with the course titles of all the courses that the instructors teach.

Comp. Sci. Instructors and Course Titles

Relational Algebra:

$$\pi_{\text{name,title}}(\sigma_{\text{dept_name}='Comp.Sci.'}(\text{instructor} \bowtie \text{teaches} \bowtie \text{course}))$$

SQL Query:

```
SELECT name, title
FROM instructor
JOIN teaches ON instructor.ID = teaches.ID
JOIN course ON teaches.course_id = course.course_id
WHERE dept_name = 'Comp.Sci.';
```

Queries on Insurance Database

Exercise: Queries on Insurance Database

Express the following queries in both Relational Algebra and SQL:

1. Find all persons who were involved in accidents in 2017.
2. Update the damage amount for car “AAB2009” in accident “AR2197” to 2000.
3. Delete all year-2010 cars belonging to the person whose ID is “12345”.

Answers

(i) Find all persons who were involved in accidents in 2017

Relational Algebra:

$$\pi_{\text{driver_id}, \text{name}, \text{address}} \left(\sigma_{\text{year}=2017}(\text{accident}) \bowtie_{\text{report_number}} \text{participated} \bowtie_{\text{driver_id}} \text{person} \right)$$

SQL:

```
SELECT DISTINCT P.driver_id, P.name, P.address
FROM person P
JOIN participated PA ON P.driver_id = PA.driver_id
JOIN accident A ON PA.report_number = A.report_number
WHERE A.year = 2017;
```

(ii) Update the damage amount for car “AAB2009” in accident “AR2197” to 2000

Relational Algebra (conceptual update):

$$\begin{aligned} \text{participated} \leftarrow & (\text{participated} - \sigma_{\text{license_plate}='AAB2009' \wedge \text{report_number}='AR2197'}(\text{participated})) \\ & \cup (\sigma_{\text{license_plate}='AAB2009' \wedge \text{report_number}='AR2197'}(\text{participated})[\text{damage_amount} := 2000]) \end{aligned}$$

SQL:

```
UPDATE participated
SET damage_amount = 2000
WHERE license_plate = 'AAB2009' AND report_number = 'AR2197';
```

(iii) Delete all year-2010 cars belonging to the person whose ID is “12345”

Relational Algebra (conceptual delete):

$$\text{car} \leftarrow \text{car} - \sigma_{\text{year}=2010 \wedge \text{license_plate} \in (\pi_{\text{license_plate}}(\sigma_{\text{driver_id}='12345'}(\text{owns})))(\text{car})$$

SQL:

```
DELETE FROM car
WHERE year = 2010
AND license_plate IN (
    SELECT license_plate
    FROM owns
    WHERE driver_id = '12345'
);
```